

Manual operativo de datos

Manual operativo de datos

Glowing Beauty S.R.L. — Automatización de recepción de facturas Departamento de Compras Septiembre 2026

1. Propósito

Este documento define **dónde se guarda cada dato y con qué forma viaja entre los componentes del sistema**. Cubre dos planos:

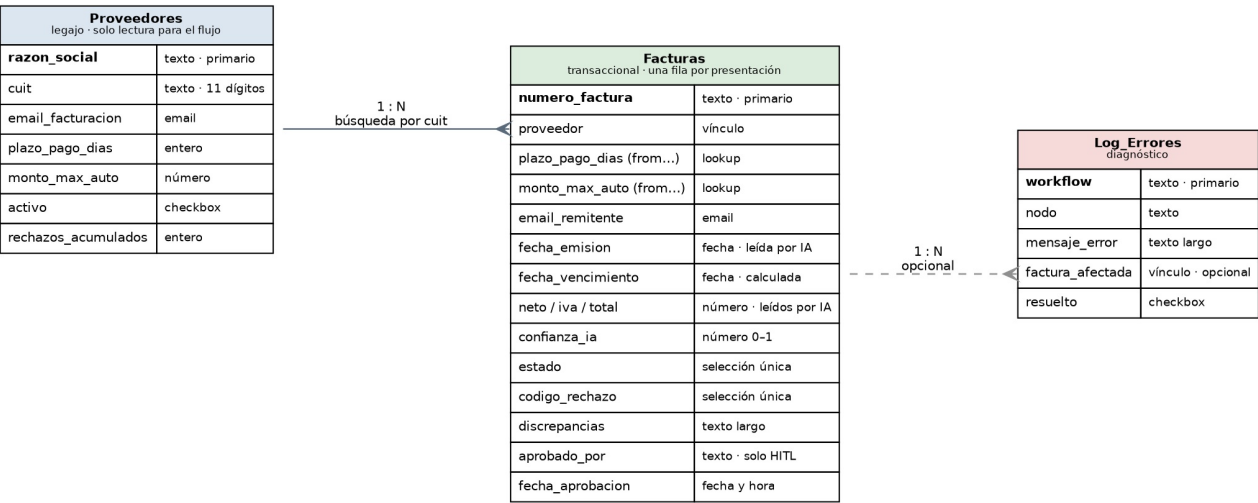
- **El modelo persistente:** las tres tablas de Airtable, sus campos, tipos y relaciones.
- **Los contratos de transferencia:** los objetos JSON que pasan de un nodo al siguiente dentro de n8n, y el contrato con el modelo de lenguaje.

La distinción importa porque el flujo maneja bastante más información de la que termina guardando. Lo que no se persiste es una decisión de diseño, no un olvido: está desarrollado en el punto 7.

2. Modelo persistente

Base de Airtable: **Glowing Beauty - Compras** (`app0STyKwMzgxNZc`). Tres tablas.

2.1 Relaciones



Modelo de datos en Airtable y sus relaciones.

Proveedores es la tabla maestra: se carga a mano desde el legajo y el flujo solo la lee. **Facturas** es transaccional: la escribe el flujo, una fila por presentación. **Log Errores** es de diagnóstico.

2.2 Tabla Proveedores

El legajo comercial. Define las dos reglas de negocio que el flujo aplica por proveedor: el plazo de pago y el tope de aprobación automática.

Campo	Tipo	Uso en el flujo
razon_social	Texto (primario)	Identificación para los mensajes
cuit	Texto, 11 dígitos	Clave de búsqueda. Se coteja contra el CUIT del asunto
email_facturacion	Email	Casilla declarada en el legajo

plazo_pago_dias	Entero (30, 60 o 90)	Sumando para la fecha de vencimiento
monto_max_auto	Número, 2 decimales	Tope de aprobación automática. Por encima, va a supervisión humana
activo	Checkbox	Baja lógica del proveedor
rechazos_acumulados	Entero	Contador de seguimiento

El CUIT se guarda **sin guiones ni espacios**, porque la búsqueda contra el asunto es por coincidencia exacta de cadena.

2.3 Tabla Facturas

Una fila por presentación recibida, incluidas las rechazadas. El rechazo también es un hecho que hay que poder auditar.

Campo	Tipo	Origen
numero_factura	Texto (primario)	Parseado del asunto: letra + 13 dígitos
proveedor	Vínculo a Proveedores	Resultado de la búsqueda por CUIT
plazo_pago_dias (from proveedor)	Lookup	Derivado del vínculo
monto_max_auto (from proveedor)	Lookup	Derivado del vínculo
email_remitente	Email	Cabecera del correo entrante
fecha_emision	Fecha	Leída por la IA del comprobante
fecha_vencimiento	Fecha	Calculada por código , no leída
neto	Número, 2 decimales	Leído por la IA
iva	Número, 2 decimales	Leído por la IA
total	Número, 2 decimales	Leído por la IA
confianza_ia	Número, 2 decimales	Declarado por el modelo
estado	Selección única	Asignado por la rama del flujo
codigo_rechazo	Selección única	Código RC activo, si lo hay
discrepancias	Texto largo	Motivo legible del rechazo o la derivación
aprobado_por	Texto	Se completa solo en la rama de supervisión humana
fecha_aprobacion	Fecha y hora (24 h)	Momento de la respuesta por Telegram

Los dos campos de lookup no se escriben: Airtable los resuelve solo a partir del vínculo. Sirven para que el dashboard muestre el tope y el plazo sin abrir la ficha del proveedor.

Importante para las actualizaciones. Los nodos que corrigen el estado tras la decisión humana escriben **solo los campos que cambian** (identificador de coincidencia, `estado`, `discrepancias`, `aprobado_por`, `fecha_aprobacion`). Un Update de Airtable con campos vacíos en el mapeo **borra** los valores ya guardados, así que el resto de las columnas se quitó deliberadamente del mapeo de esos nodos.

2.4 Tabla Log_Errores

Campo	Tipo	Contenido
workflow	Texto (primario)	Nombre del workflow, fijo
nodo	Texto	Nodo que falló, fijo por nodo de log
mensaje_error	Texto largo	Mensaje devuelto por n8n
factura_afectada	Vínculo a Facturas	Opcional, ver nota
resuelto	Checkbox	Seguimiento manual

La tabla no registra el momento del error. Es una carencia identificada, desarrollada en el punto 7.

`factura_afectada` queda vacío cuando el error ocurre en la extracción con IA: en ese punto la factura todavía no existe como registro, así que no hay nada a qué vincular. Se dejó el campo definido porque los errores de escritura en Airtable sí ocurren con el contexto disponible.

3. Catálogos

3.1 Estados de una factura

Estado	Cuándo se asigna
Aprobada	Pasó todas las validaciones bajo el tope, o la aprobó una persona
Rechazada	Activó un código RC, o la rechazó una persona
Esperando aprobacion	Derivada a supervisión humana, sin respuesta todavía
En_revision	Una persona la marcó para gestionar con el proveedor por fuera del sistema

El campo admite además `Nueva`, `Leida_IA` y `Error`, definidos al modelar la tabla y **no utilizados por el flujo actual**: el registro se crea recién al final del recorrido, ya con su estado definitivo, en vez de crearse al inicio y actualizarse en cada paso. Se conservan en el catálogo porque un diseño con escritura incremental los necesitaría.

3.2 Códigos de rechazo

Nueve códigos vigentes. El detalle de cada condición está en el *Instructivo de Presentación de Facturas*, que es el documento que reciben los proveedores.

Código	Motivo	Nodo que lo resuelve	¿Antes de la IA?
RC-01	CUIT no registrado en el legajo	Armar contexto	Sí
RC-02	Asunto con formato inválido	Validaciones de sobre	Sí
RC-03	Correo sin adjunto	Validaciones de sobre	Sí
RC-04	Más de un adjunto	Validaciones de sobre	Sí
RC-05	Formato de archivo no admitido	Validaciones de sobre	Sí
RC-06	PDF sin texto seleccionable	Validar texto PDF	Sí
RC-07	Excede el tamaño máximo	Validaciones de sobre	Sí
RC-09	Inconsistencia entre neto, IVA y total	Verificar y calcular	No
RC-10	CUIT del asunto ≠ CUIT del comprobante	Verificar y calcular	No

El rechazo por decisión humana **no lleva código**: el motivo queda registrado en `discrepancias` junto con el comentario de quien lo rechazó, porque es un juicio de negocio y no el incumplimiento de una regla del instructivo.

4. Contratos de transferencia

Dentro de `n8n`, cada nodo recibe el objeto del anterior y le agrega campos. El objeto crece de forma acumulativa a lo largo del recorrido.

4.1 Progresión del objeto

Nodo	Campos que agrega
Config	<code>telegram_chat_id</code>
Validaciones de sobre	<code>gmail_id</code> , <code>email_remitente</code> , <code>asunto</code> , <code>cuit_asunto</code> , <code>numero_factura</code> , <code>adjunto_key</code> , <code>rechazado</code> , <code>codigo_rechazo</code> , <code>motivo_rechazo</code>
Armar	<code>proveedor_id</code> , <code>razon_social</code> , <code>plazo_pago_dias</code> , <code>monto_max_auto</code>

contexto	
Validar texto PDF	texto_pdf, paginas_pdf
Verificar y calcular	cuit_emisor, razon_social_emisor, fecha_emision, fecha_vencimiento, neto, iva, total, confianza_ia, requiere_revision, supera_tope, confianza_baja, discrepancias

telegram_chat_id es un valor de entorno, no un dato de la factura: se inyecta al inicio y viaja intacto hasta el nodo de supervisión humana, que lo referencia por expresión. Es lo que evita tenerlo escrito dentro del nodo.

Los tres campos de control — rechazado, codigo_rechazo y motivo_rechazo — nacen en el segundo nodo y **se propagan respetando lo que ya venía**:

```
rechazado: ctx.json.rechazado === true || codigo !== null,
codigo_rechazo: ctx.json.codigo_rechazo || codigo,
motivo_rechazo: ctx.json.motivo_rechazo || motivo,
```

La propagación es explícita porque cada nodo inicializa su propia variable codigo en null. Escribir rechazado: codigo !== null a secas —sin consultar lo que venía— sobrescribe con false un rechazo detectado antes y la factura sigue el camino como si estuviera limpia. El punto está desarrollado en el documento de seguridad y resiliencia.

4.2 Contrato con el modelo de lenguaje

Es el único contrato que no controla el sistema de punta a punta, y por eso es el más frágil.

Lo que se le pide al modelo:

Extrae los datos de la siguiente factura argentina y devolvé únicamente un objeto JSON, sin texto adicional ni marcas de código.

Campos a devolver:

- cuit_emisor: string, solo los 11 dígitos sin guiones
- razon_social_emisor: string
- fecha_emision: string en formato AAAA-MM-DD
- neto: número
- iva: número
- total: número
- confianza: número entre 0 y 1, qué tan seguro estás de haber leído bien los importes

Reglas:

- Devolvé los números tal como figuran impresos. No calcules ni corrijas nada.
- Usá punto como separador decimal y no uses separador de miles.
- Si un campo no aparece, poné null.

La instrucción **"no calcules ni corrijas nada"** es la línea que sostiene el resto del sistema. Sin ella, un modelo que detecta que el neto más el IVA no cierran tiende a "arreglar" el total y devolver una factura coherente — que es exactamente el error que el flujo tiene que detectar. El modelo debe reportar la inconsistencia, no resolverla.

Esquema de la respuesta esperada:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "title": "Extraccion de factura",
  "type": "object",
  "required": ["cuit_emisor", "fecha_emision", "neto", "iva", "total", "confianza"],
  "properties": {
    "cuit_emisor": { "type": ["string", "null"], "pattern": "^[0-9]{11}$" },
    "razon_social_emisor": { "type": ["string", "null"] },
    "fecha_emision": { "type": ["string", "null"], "format": "date" },
    "neto": { "type": ["number", "null"], "minimum": 0 },
    "iva": { "type": ["number", "null"], "minimum": 0 },
    "total": { "type": ["number", "null"], "minimum": 0 },
    "confianza": { "type": "number", "minimum": 0, "maximum": 1 }
  }
}
```

Tolerancias implementadas. La respuesta llega como cadena dentro del campo text del nodo, no como objeto, así que hay que parsearla. Ninguna de las defensas siguientes es preventiva: cada una responde a un defecto que se materializó con algún modelo durante el desarrollo.

```
let limpio = crudo.replace(/```json/gi, '').replace(/```/g, '').trim();
const desde = limpio.indexOf('{');
const hasta = limpio.lastIndexOf('}');
if (desde !== -1 && hasta !== -1) limpio = limpio.slice(desde, hasta + 1);
limpio = limpio.replace(/:(\s*)(-?\d+(?:\.\d+)?)/g, ':$1$2');
ia = JSON.parse(limpio);
```

Defecto observado	Defensa	Modelo donde apareció
Respuesta envuelta en marcas de bloque de código	Se eliminan las marcas	Varios
Cadena de razonamiento antes o después del JSON	Se recorta desde la primera { hasta la última }	Modelos de razonamiento
Comilla de cierre en un número que nunca abrió comillas ("neto":98920.00",)	Se elimina la comilla cuando sigue a un número no entrecomillado	deepseek-v4-flash
Campo de confianza nombrado en inglés	Number(ia.confianza ?? ia.confidence) 0 acepta ambos	nemotron-3.5-lightning
Respuesta vacía o irrecuperable	Bloque catch que devuelve el objeto completo con los campos de control	dots-3-note-preview , router automático

La limpieza del recorte por llaves es la más general: no depende de qué escriba el modelo alrededor del objeto, y cubre de antemano defectos que todavía no se observaron.

El defecto del nombre del campo no es teórico: derivó **todas** las facturas a revisión por confianza cero hasta que se corrigió. El de la comilla espuria rompía el parseo por completo.

Parámetros de la llamada: modelo `inclusionai/ling-3.0-flash-fin:free`, máximo de 2.500 tokens de salida, temperatura 0. El dimensionamiento del tope está explicado en el punto 12 del documento de optimización de costos: los tokens de razonamiento interno cuentan contra el límite sin aparecer en la respuesta, así que un tope ajustado al tamaño del JSON devuelve cadena vacía.

4.3 Objeto completo tras Verificar y calcular

Ejemplo real de la ejecución de la factura 05 del test de estrés, que activó la derivación por monto:

```
{
  "gmail_id": "1a0a157a779a4d0e",
  "email_remitente": "proveedores.gb.test@gmail.com",
  "asunto": "FACTURA - 30766677788 - A0000200000903",
  "cuit_asunto": "30766677788",
  "numero_factura": "A0000200000903",
  "adjunto_key": "attachment_0",
  "rechazado": false,
  "codigo_rechazo": null,
  "motivo_rechazo": null,
  "proveedor_id": "recyz6Z00hTrLuCId",
  "razon_social": "Química Norte SRL",
  "plazo_pago_dias": 30,
  "monto_max_auto": 50000,
  "paginas_pdf": 1,
  "cuit_emisor": "30766677788",
  "razon_social_emisor": "QUIMICA NORTE S.R.L.",
  "fecha_emision": "2026-09-04",
  "fecha_vencimiento": "2026-10-05",
  "neto": 72400,
  "iva": 15204,
  "total": 87604,
  "confianza_ia": 0.95,
  "requiere_revision": true,
  "supera_tope": true,
  "confianza_baja": false,
  "discrepancias": "Total 87604 supera el tope automatico de 50000"
}
```

Se omite `texto_pdf`, que transporta el texto completo del comprobante —unos 1.800 caracteres— y es el campo que el punto 7 explica por qué no se persiste.

Lectura de este ejemplo: la factura está **limpia** (`rechazado: false`, los importes cierran, el CUIT coincide) y aun así **no se aprueba sola**, porque 87.604 supera el tope de 50.000 del proveedor. `supera_tope` en verdadero y `confianza_baja` en falso identifican cuál de los tres motivos disparó la derivación, y `discrepancias` lo dice en lenguaje

legible para quien recibe el mensaje en Telegram.

La fecha de vencimiento ilustra el cálculo: 4 de septiembre más 30 días corridos da el 4 de octubre, que en 2026 cae domingo, y por eso el sistema la corre al lunes 5.

4.4 Respuesta del formulario de Telegram

El nodo de supervisión humana devuelve un objeto acotado:

```
{
  "data": {
    "decision": "Aprobar",
    "comentario": "",
    "respondedAt": "2026-09-14T18:42:11.000Z"
  }
}
```

`decision` admite exactamente tres valores: `Aprobar`, `Rechazar` y `Marcar en revision con el proveedor`.

Esta respuesta no arrastra los datos de la factura. Es la particularidad operativa más importante del flujo: después del nodo de espera, todo dato anterior tiene que recuperarse por referencia explícita al nodo que lo produjo.

```
$('Guardar en revision').first().json.fields.<campo>
$('Verificar y calcular').first().json.<campo>
```

El mismo patrón aplica en otros dos puntos donde un nodo reemplaza el ítem en lugar de enriquecerlo: la búsqueda en *Airtable* y la extracción de texto del PDF. En ambos casos el contexto y el binario se recomponen desde nodos anteriores.

4.5 Escritura en Airtable

El mapeo del nodo que registra una aprobación:

Campo de Airtable	Expresión de origen
<code>numero_factura</code>	<code>numero_factura</code> , con el asunto como respaldo
<code>proveedor</code>	<code>{{ [\$json.proveedor_id] }}</code> — vínculo, se envía como arreglo
<code>email_remitente</code>	<code>email_remitente</code>
<code>fecha_emision</code>	<code>fecha_emision null</code>
<code>fecha_vencimiento</code>	<code>fecha_vencimiento</code>
<code>neto / iva / total</code>	Con <code> null</code>
<code>confianza_ia</code>	Con <code> null</code>
<code>estado</code>	Literal, según la rama
<code>codigo_rechazo</code>	Solo en la rama de rechazo
<code>discrepancias</code>	<code>motivo_rechazo</code> o <code>discrepancias</code>

Los `|| null` no son decorativos: el nodo de rechazo recibe datos de dos ramas distintas —la que cortó antes de la IA y la que cortó después— y en la primera los importes sencillamente no existen. Sin el respaldo, *Airtable* rechaza la escritura y el registro se pierde.

5. Identificadores

Identificador	Formato	Dónde vive
<code>numero_factura</code>	Letra + 13 dígitos (<code>A0000200000903</code>)	Clave primaria de <code>Facturas</code>
<code>cuit</code>	11 dígitos sin guiones	Clave de búsqueda contra el legajo
<code>proveedor_id</code>	<code>rec</code> + 14 caracteres	Identificador interno de <i>Airtable</i>

gmail_id	Hexadecimal	Identificador del correo, no persistente
----------	-------------	---

numero_factura se arma con las tres últimas capturas del patrón del asunto: tipo, punto de venta y número. La forma impresa en el comprobante (00002-00000903) se normaliza a la forma compacta, para que un mismo comprobante produzca siempre la misma clave.

gmail_id es el campo que el punto 7 identifica como la pieza faltante del modelo.

6. Volumetría

Magnitud	Valor
Presentaciones mensuales estimadas	400
Texto de una factura tras la extracción	~1.800 caracteres
Páginas por comprobante	1
Tamaño máximo admitido	5 MB
Registros de Facturas por año	~4.800
Proveedores en el legajo	4 en el entorno de prueba

Ninguna de estas magnitudes se acerca a los límites de Airtable ni a la ventana de contexto de los modelos evaluados. El dimensionamiento no es un criterio de decisión en este sistema.

7. Datos que el flujo maneja y no persiste

El sistema procesa más información de la que guarda. Tres campos circulan entre nodos y mueren al terminar la ejecución:

Campo	Por qué no se persiste
texto_pdf	El texto completo del comprobante, con domicilios, CUIT de terceros y detalle de productos. Es el insumo de la lectura, no un dato del proceso de aprobación. Guardarlo duplicaría en la base información que ya está en el comprobante original.
Adjunto binario	El PDF viaja por el flujo pero no se sube a Airtable. El original queda en la casilla de correo, que es el repositorio con valor documental.
gmail_id	Se calcula y se usa dentro de la ejecución, pero no se escribe.

Los dos primeros responden al principio de minimización, desarrollado en el documento de seguridad. El tercero es una carencia identificada, no una decisión.

Carencia en el registro de errores. La tabla Log_Errores no tiene campo de fecha. Un registro de diagnóstico sin marca temporal no permite distinguir un error de hoy de uno de la semana pasada, ni calcular la tasa de errores sobre una ventana de tiempo — que es justamente el KPI que el dashboard debería mostrar. La corrección es un campo de tipo hora de creación, que Airtable completa de forma retroactiva sobre los registros existentes. Está identificada y pendiente.

La pieza faltante del modelo. Persistir gmail_id resolvería dos problemas de una vez:

1. **Reprocesamiento.** El trigger de Gmail consulta los correos no leídos y no los marca como leídos al procesarlos, así que un comprobante ya procesado vuelve a ser candidato en consultas posteriores. Registrar el identificador y descartar los ya vistos corta la duplicación en la base.
2. **Presentaciones duplicadas.** Es el control que se descartó del alcance, y la misma columna lo cubriría.

La corrección está identificada y tiene un costo bajo: un campo de texto en Facturas, una búsqueda previa a la escritura. Queda fuera de esta entrega por alcance, y documentada acá y en el documento de seguridad para que la decisión sea explícita y no un descuido.

8. Resumen de responsabilidad por dato

Dato	Lo produce
Número de factura, CUIT declarado	Parseo del asunto — código
Razón social, plazo, tope	Legajo de proveedores — Airtable
Importes, fecha de emisión, CUIT impreso	Lectura del comprobante — IA
Confianza de lectura	Autoevaluación — IA
Fecha de vencimiento	Cálculo — código
Verificación aritmética, cotejo de CUIT	Comparación — código
Estado final	Ruteo — código, o decisión — persona
Aprobado por, fecha de aprobación	Supervisión humana — Telegram

El modelo aporta seis campos de los veinte que componen un registro, y ninguno de ellos determina por sí solo el resultado del proceso. Todos pasan por una verificación de código antes de convertirse en una aprobación.